

Computer programming

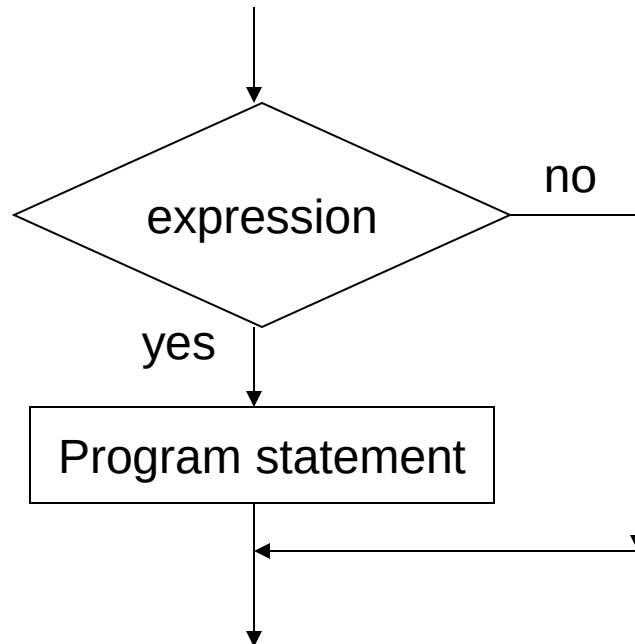
Lecture 4

Lecture 4: Outline

- **Making Decisions [chap 6 – Kochan]**
 - The `if` Statement
 - The `if-else` Construct
 - Logical Operators
 - Boolean Variables
 - Nested `if` Statements
 - The `else if` Construct
 - The `switch` Statement
 - The Conditional Operator
- **Character Input/Output**

The `if` statement

```
if ( expression )  
    program statement
```



If expression is true (non-zero), executes statement.
If gives you the choice of executing statement or skipping it.

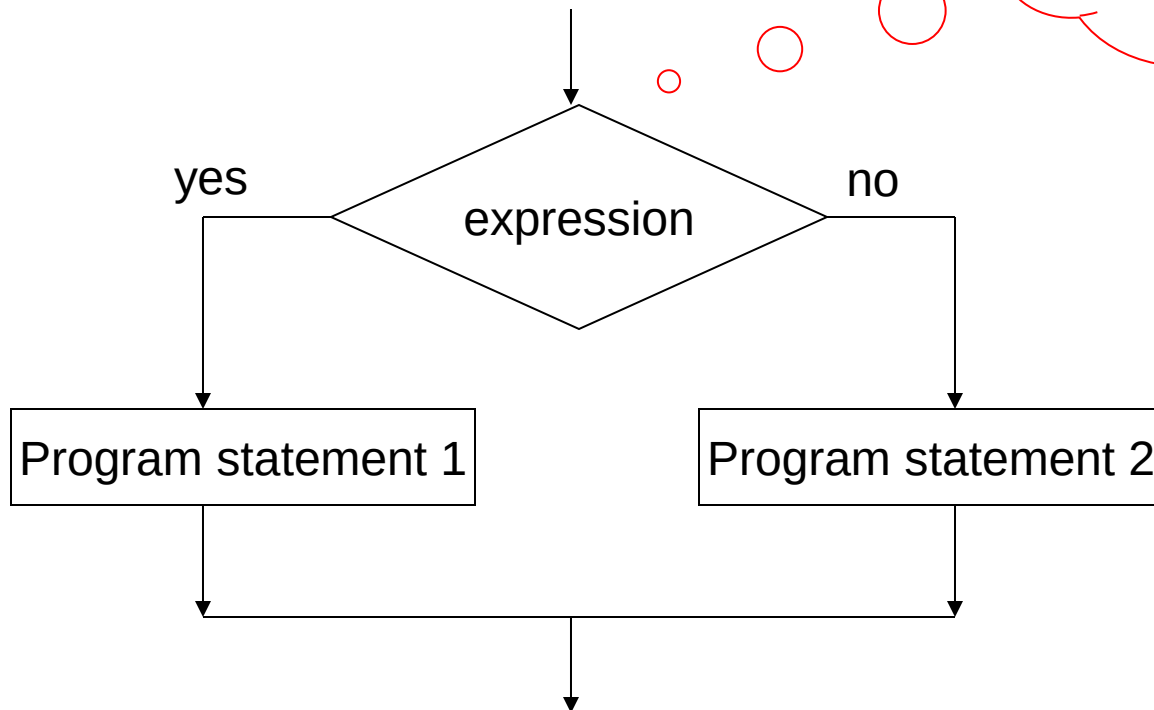
Example - if

```
// Program to calculate the absolute value of an integer
int main (void)
{
    int number;
    printf ("Type in your number: ");
    scanf ("%i", &number);
    if ( number < 0 )
        number = -number;
    printf ("The absolute value is %i\n", number);
    return 0;
}
```

The `if-else` statement

```
if ( expression )  
    program statement 1  
else  
    program statement 2
```

if-else statement:
enables you to
choose between
two statements



Example: if-else

```
// Program to determine if a number is even or odd
#include <stdio.h>
int main ()
{
    int number_to_test, remainder;
    printf ("Enter your number to be tested: ");
    scanf ("%i", &number_to_test);
    remainder = number_to_test % 2;
    if ( remainder == 0 )
        printf ("The number is even.\n");
    else
        printf ("The number is odd.\n");
    return 0;
}
```

Attention on if-else syntax !

```
if ( expression )  
    program statement 1  
else  
    program statement 2
```

In C, the ; is part
(end) of a statement !
You have to put it
also before an else
!

```
if ( remainder == 0 )  
    printf ("The number is even.\n");  
else  
    printf ("The number is odd.\n");
```

```
if ( x == 0 );  
    printf ("The number is zero.\n");
```

Syntactically OK
(void statement
on if) but
probably a
semantic error !

Example: compound relational test

```
// Program to determine if a year is a leap year
#include <stdio.h>
int main (void)
{
    int year, rem_4, rem_100, rem_400;
    printf ("Enter the year to be tested: ");
    scanf ("%i", &year);
    rem_4 = year % 4;
    rem_100 = year % 100;
    rem_400 = year % 400;
    if ( (rem_4 == 0 && rem_100 != 0) || rem_400 == 0 )
        printf ("It's a leap year.\n");
    else
        printf ("It's not a leap year.\n");
    return 0;
}
```


Logical operators

Operator	Symbol	Meaning
AND	& &	X && y is true if BOTH x and y are true
OR		X y is true if at least one of x and y is true
NOT	!	!x is true if x is false

Logical values as operands or in tests: true = non-zero, false=zero

Logical values returned as results of expressions: true = 1, false=zero

Example: 5 || 0 is 1

Example

- Program to generate a table of all prime numbers up to 50

Boolean variables

```
// Program to generate a table of prime numbers
#include <stdio.h>
int main (void) {
    int p, d;
    int isPrime;
    for ( p = 2; p <= 50; ++p ) {
        isPrime = 1;
        for ( d = 2; d < p; ++d )
            if ( p % d == 0 )
                isPrime = 0;
        if ( isPrime != 0 )
            printf ("%i ", p);
    }
    printf ("\n");
    return 0;
}
```

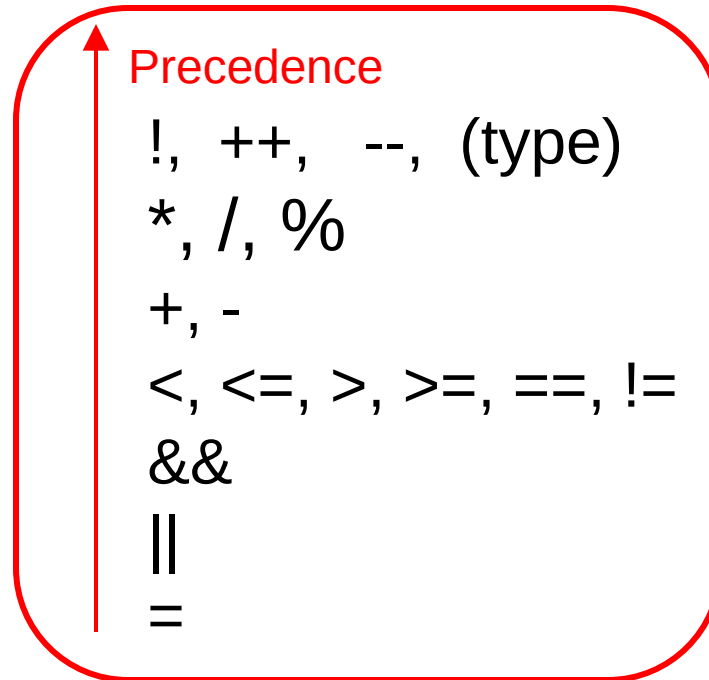
A **flag**: assumes only one of two different values. The value of a flag is usually tested in the program to see if it is “on” (TRUE) or “off” (FALSE)

Equivalent test:
(more in C-style):
if (isPrime)

Boolean variables

```
// Program to generate a table of prime numbers - rewritten
#include <stdio.h>
#include <stdbool.h>
int main (void) {
    int p, d;
    bool isPrime;
    for ( p = 2; p <= 50; ++p ) {
        isPrime = true;
        for ( d = 2; d < p; ++d )
            if ( p % d == 0 )
                isPrime = false;
        if ( isPrime )
            printf ("%i ", p);
    }
    printf ("\n");
    return 0;
}
```

Precedence of operators



Example for operator precedence:

`a > b && b > c || b > d`

Is equivalent to:

`((a > b) && (b > c)) || (b > d)`

Testing for ranges

```
if (x >= 5 && x <= 10)
    printf("in range");
```



```
if (5 <= x <= 10)
    printf("in range");
```

Testing for ranges

YES

```
if (x >= 5 && x <= 10)
    printf("in range");
```

NO!

```
if (5 <= x <= 10)
    printf("in range");
```

Syntactically correct, but semantically an error !!!

Because the order of evaluation for the `<=` operator is left-to-right, the test expression is interpreted as follows:

`(5 <= x) <= 10`

The subexpression `5 <= x` either has the value 1 (for true) or 0 (for false). Either value is less than 10, so the whole expression is always true, regardless of `x` !

Testing for character ranges

```
char ch;  
scanf("%c", &ch);  
if (ch >= 'a' && ch <= 'z')  
    printf("lowercase char\n");  
if (ch >= 'A' && ch <= 'Z')  
    printf("uppercase char\n");  
if (ch >= '0' && ch <= '9')  
    printf("digit char\n");
```

- This works for character codes such as ASCII, in which the codes for consecutive letters are consecutive numbers. However, this is not true for some codes (i.e. EBCDIC)
- A more portable way of doing this test is to use functions from `<ctype.h>`
`islower(ch)`, `isupper(ch)`, `isdigit(ch)`

Other operations on characters

```
char ch='d';  
ch=ch+1; o o o
```

c will be the
next letter 'e'

```
char ch='d';  
ch=ch+'A'-'a'; o o o
```

c will be the
corresponding
uppercase letter
'D'

- This works for character codes such as ASCII, in which the codes for consecutive letters are consecutive numbers.
- A more portable way: `<ctype.h>: toupper(c), tolower(c)`

Nested if statements

```
if (number > 5)
    if (number < 10)
        printf("1111\n");
    else printf("2222\n");
```



```
if (number > 5) {
    if (number < 10)
        printf("1111\n");
}
else printf("2222\n");
```

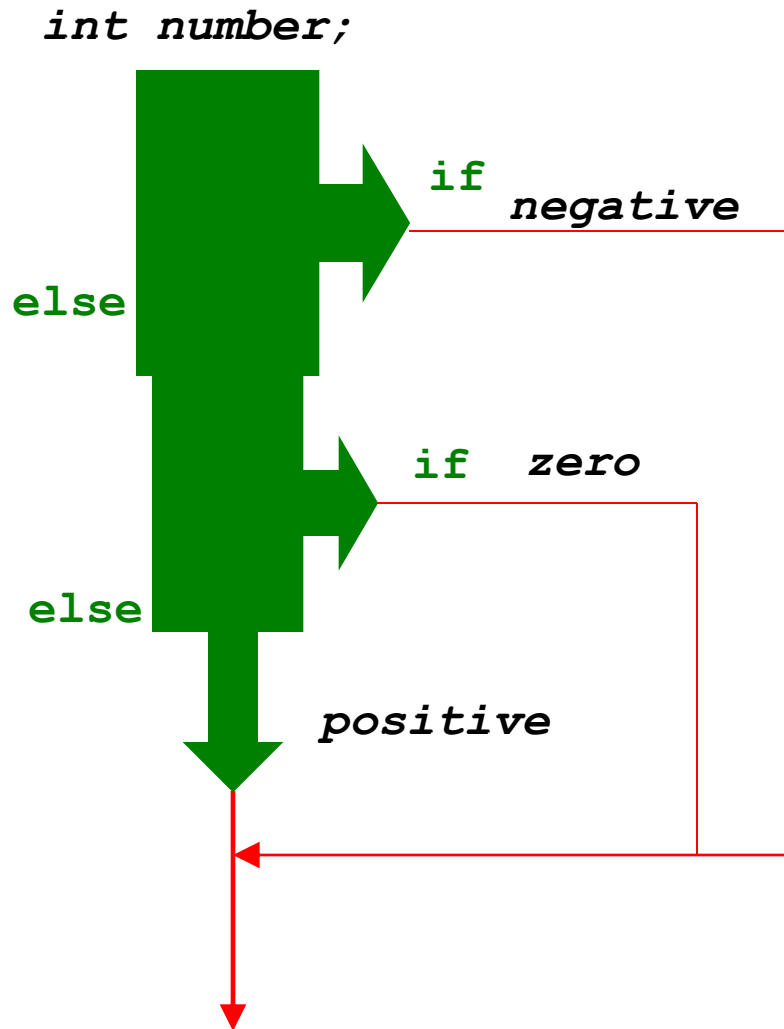


Rule: an `else` goes with the most recent `if`, unless braces indicate otherwise

Example: else-if

```
// Program to implement the sign function
#include <stdio.h>
int main (void)
{
    int number, sign;
    printf ("Please type in a number: ");
    scanf ("%i", &number);
    if ( number < 0 )
        sign = -1;
    else if ( number == 0 )
        sign = 0;
    else // Must be positive
        sign = 1;
    printf ("Sign = %i\n", sign);
    return 0;
}
```

Multiple choices – else-if



```
if ( expression 1)
    program statement 1
else if ( expression 2)
    program statement 2
else
    ○ program statement 3
    ○
    ○
```

Program style: this unindented formatting improves the readability of the statement and makes it clearer that a three-way decision is being made.

Example: else-if

```
// Program to categorize a single character
// that is entered at the terminal
#include <stdio.h>
int main (void)
{
    char c;
    printf ("Enter a single character:\n");
    scanf ("%c", &c);
    if ( (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') )
        printf ("It's an alphabetic character.\n");
    else if ( c >= '0' && c <= '9' )
        printf ("It's a digit.\n");
    else
        printf ("It's a special character.\n");
    return 0;
}
```

Example – multiple choices

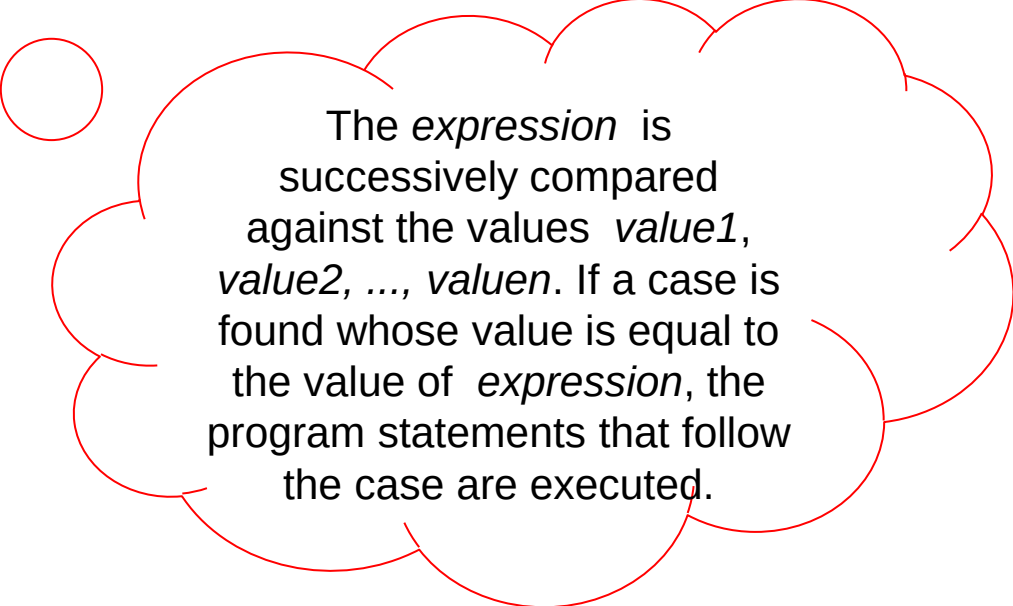
```
/* Program to evaluate simple expressions of the form
number operator number */
#include <stdio.h>
int main (void) {
    float value1, value2;
    char operator;
    printf ("Type in your expression.\n");
    scanf ("%f %c %f", &value1, &operator, &value2);
    if ( operator == '+' )
        printf (".2f\n", value1 + value2);
    else if ( operator == '-' )
        printf (".2f\n", value1 - value2);
    else if ( operator == '*' )
        printf (".2f\n", value1 * value2);
    else if ( operator == '/' )
        printf (".2f\n", value1 / value2);
    else printf ("Unknown operator.\n");
    return 0;
}
```

Example - switch

```
/* Program to evaluate simple expressions of the form
value operator value */
#include <stdio.h>
int main (void) {
    float value1, value2;
    char operator;
    printf ("Type in your expression.\n");
    scanf ("%f %c %f", &value1, &operator, &value2);
    switch (operator) {
        case '+': printf (".2f\n", value1 + value2); break;
        case '-': printf (".2f\n", value1 - value2); break;
        case '*': printf (".2f\n", value1 * value2); break;
        case '/':
            if ( value2 == 0 ) printf ("Division by zero.\n");
            else printf (".2f\n", value1 / value2);
            break;
        default: printf ("Unknown operator.\n"); break;
    }
    return 0;
}
```

The switch statement

```
switch ( expression )
{
    case value1:
        program statement
        program statement
        ...
        break;
    case value2:
        program statement
        program statement
        ...
        break;
    ...
    case valuen:
        program statement
        program statement
        ...
        break;
    default:
        program statement
        program statement
        ...
        break;
}
```



The *expression* is successively compared against the values *value1*, *value2*, ..., *valuen*. If a case is found whose value is equal to the value of *expression*, the program statements that follow the case are executed.

The switch test expression must be one with an **integer value** (including type char) (No float !).

The case values must be integer-type **constants** or integer constant expressions (You can't use a variable for a case label !)

The switch statement (cont)

Break can miss !

Statement list on
a case can miss
!

```
switch (operator)
{
    ...
    case '*':
    case 'x':
        printf (("%.2f\n", value1 * value2);
        break;
    ...
}
```

The conditional operator

condition ? expression1 : expression2

condition is an expression that is evaluated first.

If the result of the evaluation of *condition* is TRUE (nonzero), then *expression1* is evaluated and the result of the evaluation becomes the result of the operation. If *condition* is FALSE (zero), then *expression2* is evaluated and its result becomes the result of the operation

```
maxValue = ( a > b ) ? a : b;
```

Equivalent to:

```
if ( a > b )
    maxValue = a;
else
    maxValue = b;
```

Standard input/output

- The C language itself does not have any special *statements* for performing input/output (I/O) operations; all I/O operations in C must be carried out through *function calls*.
- These functions are contained in the standard C library.
- `#include <stdio.h>`
- Formatted I/O: `scanf()`, `printf()`
- Character I/O: `getchar()`, `putchar()`

Brief overview - `scanf()`

- `scanf(control string, list of arguments)`
- Control string: contains format characters
 - Important: **match** the **number** and **type** of format characters with the number and type of following arguments !
 - Format characters: as for `printf`
- Arguments: variable names prefixed with the address operator (&)
- Example:
- `scanf ("%i %i", &x, &y) ;`

How scanf works

- Scanf: searches the input stream for a value to be read, bypasses any leading *whitespace* characters
- `scanf ("%f %f", &value1, &value2);`
- `scanf ("%f%f", &value1, &value2);`
- In both cases, user can type:
 - 3 <space> 5 <enter>
 - <space> 3 <space> <space> 5 <space> <enter>
 - 3 <enter> 5 <enter>
- The exceptions: in the case of the %c format characters— the next character from the input, no matter what it is, is read
- `scanf ("%f %c %f", &value1, &op, &value2);`
 - 3 <space> + <space> 5 <enter>
 - 3 <space> + <space> <space> <enter> 5 <enter>
- `scanf ("%f%c%f", &value1, &op, &value2);`
 - 3+5<enter>
 - Not OK: 3 <space> +5<enter> => op would take the value <space>, character + would remain as input for value2 !

How scanf works

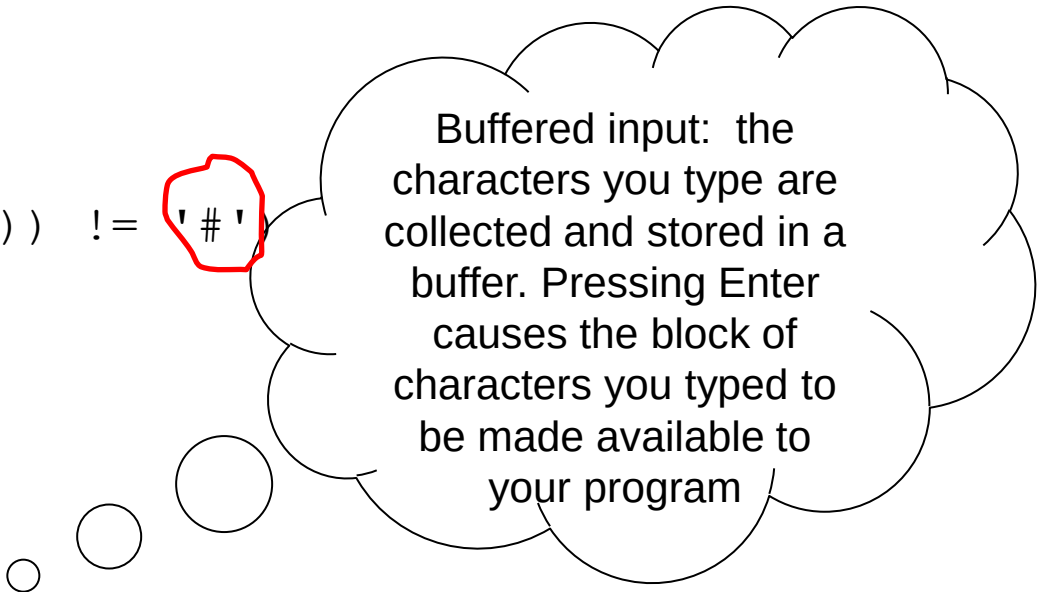
- When scanf reads in a particular value: reading of the value terminates when a character that is not valid for the value type being read is encountered.
- `scanf ("%f%c%f", &value1, &op, &value2);`
3+5<enter>
- Any nonformat characters that are specified in the format string of the scanf call are expected on the input.
- `scanf ("%i:%i:%i", &hour, &minutes, &seconds);`
3:6:21<enter>
3<space>:<space>6<space>:<space>21<enter>
3<space>6<space>21<space> => NOT OK !
- The next call to scanf picks up where the last one left off.
- `scanf ("%f", &value1);`
- User types: 7 <space> 8 <space> 9 <enter>
- 7 is stored in `value1`, rest of the input chars remain waiting in buffer
- `scanf ("%f", &value2);`
- 8 from buffer is stored in `value2`, rest of the input remains waiting

getchar () and putchar ()

- The simplest input mechanism is to read one character at a time from the *standard input*, with `getchar`
- To display a character: `putchar`

Example: getchar()

```
#include <stdio.h>
int main(void) {
    char ch;
    while ((ch = getchar()) != '#')
        putchar(ch);
    return 0;
}
```

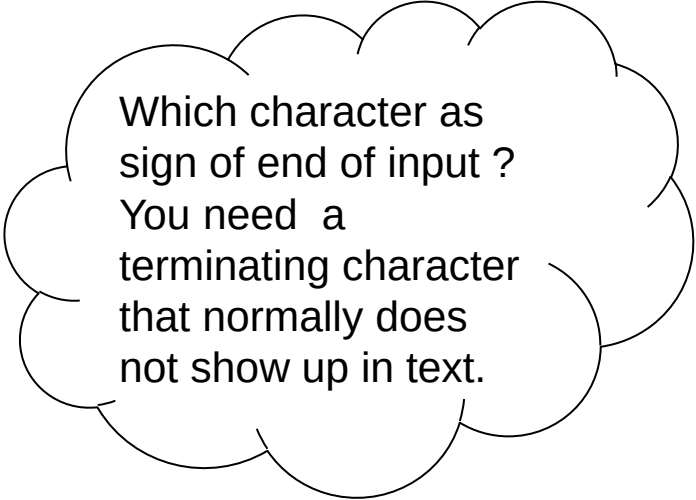


Buffered input: the characters you type are collected and stored in a buffer. Pressing Enter causes the block of characters you typed to be made available to your program

```
Hello ! I am<enter>
Hello ! I am
Joe from #3.<enter>
Joe from
```


Terminating keyboard input

```
#include <stdio.h>
int main(void) {
    int ch;
    while ((ch = getchar()) != EOF)
        putchar(ch);
    return 0;
}
```



Which character as
sign of end of input ?
You need a
terminating character
that normally does
not show up in text.

getchar returns the next input character each time it is called,
or EOF when it encounters end of file.

EOF is a symbolic constant defined in <stdio.h>. (The value is typically -1)

EOF from the keyboard: Ctrl+Z

Exercise: getchar()

```
/* Read characters from input over several lines until EOF.  
   Count lines and characters in input */
```

```
#include <stdio.h>  
int main(void) {  
    int  c,  nl, nc;  
    nl = 0;  
    nc = 0;  
    while ((c = getchar()) != EOF) {  
        nc++;  
        if (c == '\n')  
            nl++;  
    }  
    printf("Number of lines in input: %d\n",  nl);  
    printf("Number of characters in input: %d\n",  nc);  
    return 1;  
}
```